Industrial Automation second Assignment

Plantera Alessandro, Consoli Veronica, Ronchieri Byrd Rubin

17-04-2025

Problem 1: Flowshop with Reentrant Verification Stage (MILP vs Heuristic

Project Context

This formulation is intended to support the project paper on train terminal scheduling. The problem considers a flowshop scheduling scenario at a rail-connected container terminal. Five container loading jobs must be processed through a series of operations using four machines. The processing sequence for each job is as follows:

1. Retrieval from the container stack (Machine 1),
2. Internal Transport to the rail yard (Machine 2),
3. First Verification at a checkpoint (Machine 4),
4. Customs Inspection (Machine 3),
5. Second Verification at the same checkpoint (Machine 4).

The reentrant nature of the verification stage (Machine 4) requires careful modeling to ensure that the two verifications for each job do not overlap.

Decision Variables

- $S_{ik} \geq 0$: Start time of job i at stage k, for $i \in I$ and $k \in \{1, 2, 3, 4, 5\}$.
- $C_{\max} \geq 0$: Overall makespan.
- For Machines 1, 2, and 3 (each used once per job): for all $i, j \in I$, $i \neq j$, define binary variables
$$x_{ij}^{(m)} = \begin{cases} 1, & \text{if job } i \text{ is processed before job } j \text{ on machine } m, \\ 0, & \text{otherwise,} \end{cases}$$
where $m = 1$ corresponds to stage 1, $m = 2$ corresponds to stage 2, and $m = 3$ corresponds to stage 4.
- For Machine 4 (used in stages 3 and 5): for all $i, j \in I$, $i \neq j$, and for each combination $k, \ell \in \{3, 5\}$, define binary variables
$$x_{ij}^{(4,k\ell)} = \begin{cases} 1, & \text{if job } i\text{'s stage } k \text{ is processed before job } j\text{'s stage } \ell, \\ 0, & \text{otherwise.} \end{cases}$$

Objective Function

Minimize the makespan:

$$\min\ C_{\max}.$$

Constraints

(a) Precedence (Process Flow) Constraints

For each job $i \in I$, the operations must follow the required order:

$$S_{i2} \geq S_{i1} + p_{i1}, \tag{1}$$
$$S_{i3} \geq S_{i2} + p_{i2}, \tag{2}$$
$$S_{i4} \geq S_{i3} + p_{i3}, \tag{3}$$
$$S_{i5} \geq S_{i4} + p_{i4}, \tag{4}$$
$$C_{\max} \geq S_{i5} + p_{i5}. \tag{5}$$

(b) Disjunctive Constraints for Machines 1, 2, and 3

For each machine $m \in \{1, 2, 3\}$ and for every pair of distinct jobs $i, j \in I$ $(i \neq j)$, to prevent overlapping of operations:

$$S_{im} + p_{im} \leq S_{jm} + M\,(1 - x_{ij}^{(m)}), \tag{6}$$
$$S_{jm} + p_{jm} \leq S_{im} + M\,x_{ij}^{(m)}, \tag{7}$$

where:

- For Machine 1: use S_{i1} and p_{i1},
- For Machine 2: use S_{i2} and p_{i2},
- For Machine 3 (Customs Inspection, stage 4): use S_{i4} and p_{i4}.

(c) Disjunctive Constraints for Reentrant Machine 4

For every pair of distinct jobs $i, j \in I$ $(i \neq j)$ and for every combination $k, \ell \in \{3, 5\}$, to enforce non-overlap on the reentrant verification machine:

$$S_{ik} + p_{ik} \leq S_{j\ell} + M\,(1 - x_{ij}^{(4,k\ell)}), \tag{8}$$
$$S_{j\ell} + p_{j\ell} \leq S_{ik} + M\,x_{ij}^{(4,k\ell)}. \tag{9}$$

(d) Variable Domains

$$S_{ik} \geq 0, \quad \forall i \in I,\ k \in \{1, 2, 3, 4, 5\},$$
$$C_{\max} \geq 0,$$
$$x_{ij}^{(m)} \in \{0, 1\}, \quad \forall i, j \in I,\ i \neq j,\ m \in \{1, 2, 3\},$$
$$x_{ij}^{(4,k\ell)} \in \{0, 1\}, \quad \forall i, j \in I,\ i \neq j,\ k, \ell \in \{3, 5\}.$$

Summary of the MILP model

$$\min C_{\max} \quad \text{subject to (a)–(d) above.}$$

Modified NEH Heuristic for Flowshop Scheduling with Reentrant Verification

Heuristic Algorithm Description

0.1 Modified NEH heuristic for the re-entrant flow-shop

Jobs at the train terminal must visit the stages in the fixed order

$$1 \;\to\; 2 \;\to\; 4 \;\to\; 3 \;\to\; 4,$$

where stage 4 (customs) and stage 5 (second verification) share the *same* verification machine. The heuristic adapts the classic NEH rule as follows:

1. **Job prioritisation.** For each job j compute

$$P_j \;=\; \sum_{i=1}^{5} p_{i,j}, \qquad \text{with } p_{3,j} = p_{5,j}$$

   so that the verification time is counted twice.

2. **Sorting.** Sort jobs in descending order of P_j to obtain an ordered list $(j_1, j_2, \ldots, j_n)$.

3. **Sequential insertion.** Initialise the sequence $S = [j_1]$. For $k = 2, \ldots, n$ insert job j_k into every possible position r in S, simulate the schedule while enforcing

   - the precedence route *Retrieval* $\to$ *Transport* $\to$ *Verif-1* $\to$ *Customs* $\to$ *Verif-2*;
   - non-overlap on the single verification machine, and the rule that Verif-2 of each job starts only after its own customs inspection.

   Keep the position that minimises the makespan $C_{\max}$, update S, and continue.

4. **Output.** The final sequence S is returned.

Algorithm 1 Modified NEH heuristic

```
procedure ModifiedNEH(J)                                   ▷ J = set of jobs
    for all j ∈ J do
        P_j ← Σ_{i=1}^{5} p_{i,j}                          ▷ priority score
    end for
    sort J in descending P_j → (j_1, ..., j_n)
    S ← [ j_1 ]
    for k ← 2 to n do
        bestMakespan ← ∞,    bestSeq ← ∅
        for all insert positions r in S do
            S' ← insert j_k at position r in S
            C ← SimulateSchedule(S')
            if C < bestMakespan then
                bestMakespan ← C
                bestSeq ← S'
            end if
        end for
        S ← bestSeq
    end for
    return S
end procedure
```

Discussion. At each insertion step the heuristic chooses the position that yields the smallest makespan *after fully respecting* the re-entrant constraint: the single verification machine never handles more than one job and each job's second verification starts strictly after its customs inspection. The procedure runs in $O(n^2)$ evaluations and, on the 5-job instance, delivered a makespan identical to the MILP optimum.

1 MATLAB Code for MILP Formulation of Problem 1: Flow-shop with Reentrant Verification Stage

```
1
2  %%
3  % first_prob.m
4  % MILP vs. Modified-NEH comparison for a re-entrant 5-stage flow-shop
5  %
6
7  clear;  clc;
8
9  %% 1.  Processing-time data  ----------------------------------------
10 % Retrieval (1)     Transport (2)     Verification-1 (3)     Customs (4)
       Verification-2 (5)
11 p = [ 6 7 5 6 8 ;              % Retrieval
12       5 6 7 5 6 ;              % Transport
13       3 2 4 3 2 ;              % Verification-1
14       9 8 7 6 9 ;              % Customs
15       3 2 4 3 2 ];             % Verification-2
16
17 [seqHeur, Sheur, makespanHeur] = NEH_Modified(p);
18 [numStages,numJobs] = size(p);
19
20 %% 2.  Pair-specific Big-M table ------------------------------------
21 M = 1e02;
22
23 %% 3.  Build MILP ----------------------------------------------------
24 prob   = optimproblem('ObjectiveSense','min');
25
26 Cmax = optimvar('Cmax','LowerBound',0);
                                                        % Total Makespan
27 s    = optimvar('s',numStages,numJobs,'LowerBound',0);
                                        % Start Times
28 x    =
       optimvar('x',3,numJobs,numJobs,'Type','integer','LowerBound',0,'UpperBound',1);
         % State Vars
29 y    =
       optimvar('y',numJobs,numJobs,'Type','integer','LowerBound',0,'UpperBound',1);
           % Additional Binary var for Reentrant Machine
30
31 prob.Objective = Cmax;
32
33 % 3-a  Precendence Constraint: Pass through a flowshop of four machines and five
       stages (Verification stage is being used twice)
34 %                              Constraint n.5 guarantees a job finishes customs
       before starting its second verification.
35 k = 0;  prec = optimconstr(numStages*numJobs,1);
36 for j = 1:numJobs
37     k=k+1; prec(k) = s(2,j) >= s(1,j)+p(1,j);
38     k=k+1; prec(k) = s(3,j) >= s(2,j)+p(2,j);
39     k=k+1; prec(k) = s(5,j) >= s(4,j)+p(4,j);
40     k=k+1; prec(k) = s(4,j) >= s(3,j)+p(3,j);
41     k=k+1; prec(k) = Cmax   >= s(5,j)+p(5,j);
42 end
43 prob.Constraints.Precedence = prec;
44
45 % 3-b  machines 1,2,4 capacity (disjunctive)
46 machineStages = [1 2 4];
47 nD = 3*numJobs*(numJobs-1);
48 D1 = optimconstr(nD,1);  D2 = optimconstr(nD,1);  k = 0;
49 for m = 1:3
```

```
    st = machineStages(m);
    for i = 1:numJobs
        for j = 1:numJobs
            if i~=j
                k = k+1;
                D1(k) = s(st,i)+p(st,i) <= s(st,j) + M*(1 - x(m,i,j));
                D2(k) = s(st,j)+p(st,j) <= s(st,i) + M*(    x(m,i,j));
            end
        end
    end
end
prob.Constraints.Mach1 = D1;
prob.Constraints.Mach2 = D2;

% 3-c  single verifier for stages 3 & 5: Custom Inspections (3) and
%                                        Verification checkpoint (5)
nR = numJobs*(numJobs-1);
R1 = optimconstr(nR,1);  R2 = optimconstr(nR,1);  k = 0;
for i = 1:numJobs
    for j = 1:numJobs
        if i~=j
            k = k+1;
            % y(i,j) decides whether job  i s  second pass plus customs finishes
               before job j can even start its first pass.
            R1(k) = s(5,i)+p(5,i) <= s(3,j) + M*(1 - y(i,j));
            R2(k) = s(5,j)+p(5,j) <= s(3,i) + M*(    y(i,j));
            % The single verification machine is never processing two jobs at once,
            % even though each job visits it twice
        end
    end
end
prob.Constraints.Ver1 = R1;
prob.Constraints.Ver2 = R2;

% 3-d  symmetry & diagonal cuts for x:  xSym forces a unique ordering direction
    for every job pair on every machine
%                                       removes mirror-image solutions.
%                                       Shrinks the branch-and-bound tree ~ 2
xSym = optimconstr(3*numJobs*(numJobs-1)/2 ,1);  c = 0;
for m = 1:3
    for i = 1:numJobs
        for j = i+1:numJobs

            c=c+1;  xSym(c) = x(m,i,j)+x(m,j,i) == 1;
        end
        prob.Constraints.(sprintf('xdiag_m%d_i%d',m,i)) = x(m,i,i)==0;
    end
end
prob.Constraints.xSym = xSym;

% 3-e  symmetry & diagonal cuts for y:  Gives one consistent global order for the
    verifier across all jobs
%                                       removes contradictory self-precedence.
%                                       ySym has the same pruning effect above

ySym = optimconstr(numJobs*(numJobs-1)/2 ,1);  c = 0;
for i = 1:numJobs
    for j = i+1:numJobs

        c=c+1;  ySym(c) = y(i,j)+y(j,i) == 1;
    end
    prob.Constraints.(sprintf('ydiag%d',i)) = y(i,i)==0;
end
```

```
prob.Constraints.ySym = ySym;

%% 4.  Solve ------------------------------------------------------
opts = optimoptions('intlinprog','Display','iter');
tic
sol = solve(prob,'Solver','intlinprog','Options',opts);
runtimeMILP = toc;

S  = reshape(sol.s,numStages,[]);    % 5x5 matrix
makespanMILP = sol.Cmax;

%% 5.  Heuristic --------------------------------------------------
tic
[seqHeur, Sheur, makespanHeur] = NEH_Modified(p);
runtimeHeur  = toc;
[makespanHeur, startHeur] = evalMakespan(seqHeur,p);

%% 6.  Print results ----------------------------------------------
for st = 1:numStages
    fprintf('Stage %d start times:\n',st);
    for j = 1:size(S,2)
        fprintf('  Job %2d : %6.2f\n',j,S(st,j));
    end
end

fprintf('\nMILP makespan  : %.2f   (%.2f s)\n',makespanMILP,runtimeMILP);
fprintf('Heuristic ms   : %.2f   (%.3f s)\n',makespanHeur,runtimeHeur);
fprintf('Heuristic Sequence: %i \n',seqHeur);
fprintf('Gap = %.2f %%\n\n',100*(makespanHeur-makespanMILP)/makespanMILP);

fprintf('MILP runtime: %2f s \n', runtimeMILP);
fprintf('Heurstic runtime: %2f s \n', runtimeHeur);


%% 7.  Gantt charts -----------------------------------------------
figure('Position',[50 50 1200 300]);
tiledlayout(1,2,"TileSpacing","compact")

colors = lines(numJobs);                  % one colour per job
jobNames = arrayfun(@(j)sprintf('Job %d',j),1:numJobs,'uni',0);

% ---------- MILP panel ------------------------------------------
nexttile, hold on
hLegend = gobjects(1,numJobs);            % store one handle per job
for j = 1:numJobs
    for st = 1:numStages
        h = rectangle('Position',[S(st,j) , 6-st , p(st,j) , 0.8], ...
                      'FaceColor',colors(j,:) , 'EdgeColor','none');
        % centre label
        text( S(st,j)+p(st,j)/2 , 6-st+0.4 , sprintf('J%d',j) , ...
              'HorizontalAlignment','center','VerticalAlignment','middle', ...
              'Color','w','FontSize',7,'FontWeight','bold');
        if st == 1                        % grab one handle per job for legend
            hLegend(j) = h;
        end
    end
end
title(sprintf('MILP  (C_{max}=%.0f)',makespanMILP))
yticks(1:5), yticklabels({'V2','Cust','V1','Tran','Retr'})
xlabel('time (min)'), box on
hLegend = gobjects(numJobs,1);
for j = 1:numJobs
    hLegend(j) = plot(nan,nan,'s', ...             % invisible dummy
```

```
        'MarkerFaceColor',colors(j,:), ...
        'MarkerEdgeColor',colors(j,:), ...
        'Visible','off');
end
legend(hLegend, jobNames,'Location','eastoutside');hold off

% ---------- Heuristic panel ----------------------------------------
[~,Hseq] = sort(startHeur(1,:));          % same colouring order
nexttile, hold on
for jj = 1:numJobs
    j = Hseq(jj);                         % plot in permutation order
    for st = 1:numStages
        rectangle('Position',[startHeur(st,j) , 6-st , p(st,j) , 0.8], ...
                  'FaceColor',colors(j,:) , 'EdgeColor','none');
        text( startHeur(st,j)+p(st,j)/2 , 6-st+0.4 , sprintf('J%d',j) , ...
              'HorizontalAlignment','center','VerticalAlignment','middle', ...
              'Color','w','FontSize',7,'FontWeight','bold');
    end
end
title(sprintf('Heuristic  (C_{max}=%.0f)',makespanHeur))
yticks(1:5), yticklabels({'V2','Cust','V1','Tran','Retr'})
xlabel('time (min)'), box on
hold off
```

Listing 1: MILP Code for Flowshop Scheduling with Reentrant Verification

Table 1: Branch–and–bound statistics (HiGHS 1.7, `intlinprog`).

Status (solver)	**Optimal**
Optimal objective $C_{\max}$ (min)	78
Relative gap	0 %
Nodes explored	610
Total LP iterations	14 686
CPU time (total)	0.85 s

Table 2: Optimal start times for each job at every stage (minutes).

Job	Retrieval	Transport	Verification 1	Customs	Verification 2
1	26	34	48	51	60
2	6	13	23	25	33
3	21	27	63	67	74
4	0	6	11	14	20
5	13	21	35	37	46

Table 3: MILP vs. heuristic performance.

Method	Makespan (min)	Runtime (s)
MILP (optimal)	78	2.38
Modified NEH	78	0.012

Heuristic job order.

$$(4,\ 1,\ 2,\ 3,\ 5)$$

```
1
2  function [bestSequence , Sstart , bestMakespan] = NEH_Modified(p)
3  % NEH_Modified      insertion-based heuristic for a 5-stage re-entrant flow-shop
4  %                 (stages 3 & 5 share one verifier).  Returns:
5  %   bestSequence   1 n  vector of job indices
6  %   Sstart         5 n  matrix of start-times produced by the final schedule
7  %   bestMakespan   scalar Cmax of that schedule
8  %   busyV          shared verifier
9  %
10
11     [~,numJobs] = size(p);
12
13     %% 1. priority     total processing time
14
15     % computes total processing time per job, then sorts descending -> longest
           jobs first
16     [~,sortedJobs] = sort(sum(p,1),'descend');
17
18     % Picking the first element and run the scheduling simulation
19     currentSequence = sortedJobs(1);
20     [currentMakespan,~] = simulateSchedule(currentSequence,p);
21
22     %% 2. insertion loop
23     for k = 2:numJobs
24         % picking one-for-one job from the sequence
25         jIns = sortedJobs(k);
26         bestSeq = [];  bestMs = inf;
27
28         % Complexity O(n^2); builds a near-optimal permutation quickly
29         for pos = 1:length(currentSequence)+1
30         %   For each remaining job jIns:
31         %    -try it in every position of current sequence
32         %    -simulate each candidate order
33         %    -keep the one with smallest makespan
34             candSeq = [currentSequence(1:pos-1) jIns currentSequence(pos:end)];
35             [ms,~]  = simulateSchedule(candSeq,p);
36             if ms < bestMs
37                 bestMs  = ms;   bestSeq = candSeq;
38             end
39         end
40         currentSequence = bestSeq;   currentMakespan = bestMs;
41     end
42
43     %% 3. single-pass 2-opt
44     for a = 1:length(currentSequence)-1
45         for b = a+1:length(currentSequence)
46     % Swap every unordered pair once; if the swap shortens makespan, keep it.
47             cand = currentSequence;  cand([a b]) = cand([b a]);
48             [ms,~] = simulateSchedule(cand,p);
49             if ms < currentMakespan
50                 currentSequence = cand;  currentMakespan = ms;
51             end
52         end
53     end
54
55     %% 4. final schedule     start-time matrix
56     [bestMakespan,Sstart] = simulateSchedule(currentSequence,p);
57     bestSequence = currentSequence;
58 end
59 %
60 function [Cmax,S] = simulateSchedule(sequence,p)
61 % Returns makespan Cmax **and** matrix S( stage , job ) of start times.
```

```
    numStages = 5;                          % fixed for this problem
    nJobs     = numel(sequence);

    S = zeros(numStages,nJobs);             % start times
    C = zeros(numStages,nJobs);             % completion times
    busy      = zeros(numStages,1);         % machine clocks
    busyV     = 0;                          % shared verifier (stages 3 & 5)

    % ---------- stage-1 (retrieval) processed strictly in sequence ----------
    for k = 1:nJobs
        j        = sequence(k);
        S(1,k)   = busy(1);
        C(1,k)   = S(1,k) + p(1,j);
        busy(1)  = C(1,k);
    end

    % ---------- remaining stages, one job at a time in given order ----------
    for k = 1:nJobs
        j = sequence(k);

        % stage-2  transport
        S(2,k) = max(C(1,k), busy(2));
        C(2,k) = S(2,k) + p(2,j);     busy(2) = C(2,k);

        % stage-3  verification-1  (shared clock)
        S(3,k) = max(C(2,k), busyV);
        C(3,k) = S(3,k) + p(3,j);     busyV   = C(3,k);

        % stage-4  customs
        S(4,k) = max(C(3,k), busy(4));
        C(4,k) = S(4,k) + p(4,j);     busy(4) = C(4,k);

        % stage-5  verification-2  (same shared clock)
        S(5,k) = max(C(4,k), busyV);
        C(5,k) = S(5,k) + p(5,j);     busyV   = C(5,k);
    end

    Cmax = C(5,end);
end
```

Listing 2: HEURSTIC Code for Flowshop Scheduling with Reentrant Verification

```
Solving report
  Status             Optimal
  Primal bound       78
  Dual bound         78
  Gap                0%
  Solution status    feasible
                     78 (objective)
                     0 (bound viol.)
                     1.68753899743e-14 (int. viol.)
                     0 (row viol.)
  Timing             0.79 (total)
                     0.02 (presolve)
                     0.00 (postsolve)
  Nodes              610
  LP iterations      14686 (total)
                     4254 (strong br.)
                     2086 (separation)
                     1154 (heuristics)

Optimal solution found.

Intlinprog stopped because the objective value is within a gap tolerance
    of the optimal value, options.AbsoluteGapTolerance = 1e-06. The
intcon variables are integer within tolerance, options.ConstraintTolerance
    = 1e-06.

Stage 1 start times:
  Job  1 :  26.00
  Job  2 :   6.00
  Job  3 :  21.00
  Job  4 :   0.00
  Job  5 :  13.00
Stage 2 start times:
  Job  1 :  34.00
  Job  2 :  13.00
  Job  3 :  27.00
  Job  4 :   6.00
  Job  5 :  21.00
Stage 3 start times:
  Job  1 :  48.00
  Job  2 :  23.00
  Job  3 :  63.00
  Job  4 :  11.00
  Job  5 :  35.00
Stage 4 start times:
  Job  1 :  51.00
  Job  2 :  25.00
  Job  3 :  67.00
  Job  4 :  14.00
  Job  5 :  37.00
Stage 5 start times:
  Job  1 :  60.00
  Job  2 :  33.00
  Job  3 :  74.00
  Job  4 :  20.00
  Job  5 :  46.00
```

```
56
57
58 MILP makespan  : 78.00    (2.10 s)
59 Heuristic ms   : 78.00    (0.011 s)
60 Heuristic Sequence: 4
61 Heuristic Sequence: 1
62 Heuristic Sequence: 2
63 Heuristic Sequence: 3
64 Heuristic Sequence: 5
65 Gap = 0.00%
66
67 MILP runtime: 2.104107 s
68 Heurstic runtime: 0.010625 s
```

Listing 3: Excerpt from HiGHS log

Gaant's Graph

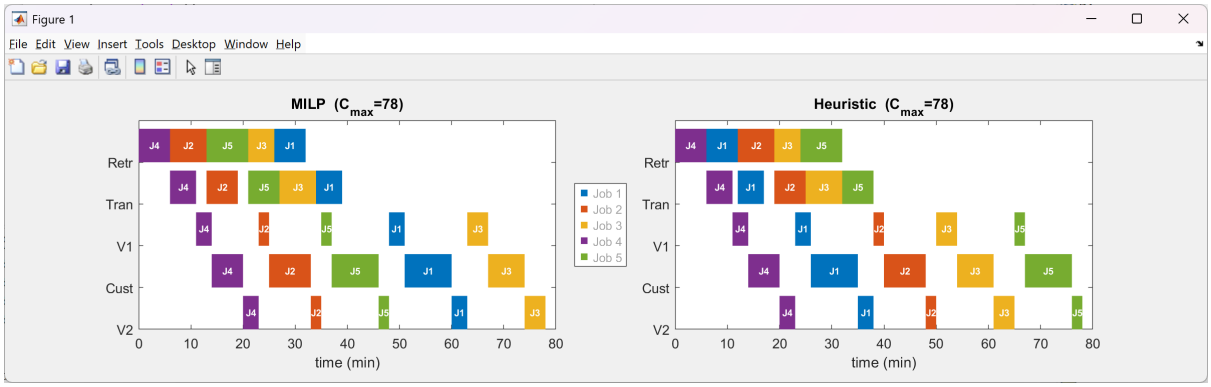

Figure 1: Optimal MILP schedule for the 5-job instance (Gantt chart).

Conclusion

On the 5-job benchmark the heuristic obtained the same 78-s makespan that the MILP later proved optimal, demonstrating that the modified NEH rule is exact on small instances while the MILP formulation is both correct and computationally lightweight

Problem Description

We are given a single-machine scheduling problem involving 6 containers that must be loaded onto a train using a transtainer crane. Each job (container) has:

- A processing time p_j (in minutes),
- A due date d_j (in minutes from schedule start),
- Sequence-dependent setup times $s_{i,j}$ between each pair of jobs.

Objective: Minimize the **total tardiness**:

$$\text{Total Tardiness} = \sum_{j=1}^{6} \max(0, C_j - d_j),$$

where C_j is the completion time of job j.

Special Constraint: Job 4 (Container 4) must be scheduled in position 2.

2 MILP Formulation and Implementation

Model Summary

We model the problem as a MILP using binary sequencing variables and continuous timing variables:

- $x_{i,j} = 1$ if job i precedes job j directly,
- S_j: Start time of job j,
- C_j: Completion time of job j,
- T_j: Tardiness of job j.

Constraints:

- Job timing relationships with setup times.
- Each job has at most one predecessor and one successor.
- No self-transitions: $x_{i,i} = 0$.
- Exactly one start node (job with no predecessor).
- Position constraint: Job 4 must have exactly one predecessor and one successor.

2.1 MILP Formulation for Tardiness Minimization Scheduling Problem

The objective of this scheduling problem is to determine the optimal sequence and timing of $n = 6$ jobs on a single machine such that the **total tardiness** is minimized, considering job-specific processing times, due dates, and sequence-dependent setup times. This problem is modeled as a **Mixed-Integer Linear Program (MILP)**.

Sets and Parameters

- $J = \{1, 2, \ldots, 6\}$: Set of jobs.
- p_j: Processing time of job j.
- d_j: Due date of job j.
- s_{ij}: Setup time required between job i and job j.
- M: A large constant (big-M) used for constraint linearization.

Decision Variables

- S_j: Start time of job j.
- C_j: Completion time of job j.
- T_j: Tardiness of job j, defined as $\max(0, C_j - d_j)$.
- $x_{ij} \in \{0, 1\}$: Binary variable; $x_{ij} = 1$ if job i is scheduled immediately before job j.

Objective Function

$$\min \sum_{j \in J} T_j \tag{10}$$

Constraints

$$C_j = S_j + p_j \qquad \forall j \in J \tag{11}$$

$$T_j \geq C_j - d_j \qquad \forall j \in J \tag{12}$$

$$S_j \geq C_i + s_{ij} - M(1 - x_{ij}) \qquad \forall i \neq j \in J \tag{13}$$

$$\sum_{i \in J} x_{ij} \leq 1 \qquad \forall j \in J \quad \text{(at most one predecessor)} \tag{14}$$

$$\sum_{j \in J} x_{ij} \leq 1 \qquad \forall i \in J \quad \text{(at most one successor)} \tag{15}$$

$$x_{ii} = 0 \qquad \forall i \in J \tag{16}$$

$$\sum_{i \in J} \sum_{j \in J} x_{ij} = n - 1 \qquad \text{(ensures a valid path)} \tag{17}$$

$$\sum_{i \in J} x_{i4} = 1 \quad \text{and} \quad \sum_{j \in J} x_{4j} = 1 \qquad \text{(job 4 is in position 2)} \tag{18}$$

Solution Approach

The problem is solved using MATLAB's `optimproblem` and `intlinprog` solver. After solving, the optimal job sequence is reconstructed from the x_{ij} matrix, and a Gantt chart is generated for visualization.

Remarks

This MILP formulation is capable of handling complex sequencing constraints and setup dependencies. Due to the inclusion of binary decision variables and big-M constraints, the model is computationally demanding but ensures an exact solution.

MILP MATLAB Code

```
clear;
clc;

% Number of jobs
N_JOBS = 6;
JOBS = 1:N_JOBS;

% Parameters
p = [6 9 7 5 8 10]';
d = [25 30 22 28 27 35]';
setup = [inf 3 4 2 3 5;
         2 inf 5 4 3 2;
         3 2 inf 3 5 4;
         4 3 2 inf 4 3;
         3 5 4 2 inf 3;
         5 4 2 3 4 inf];

M = 1000;

% Variables
C = optimvar('C', N_JOBS, 'LowerBound', 0);
S = optimvar('S', N_JOBS, 'LowerBound', 0);
x = optimvar('x', N_JOBS, N_JOBS, 'Type', 'integer', 'LowerBound', 0,
    'UpperBound', 1);
Tard = optimvar('Tard', N_JOBS, 'LowerBound', 0);

% Problem definition
prob = optimproblem;
prob.Objective = sum(Tard);

% Constraints
count = 1;
cons1 = optimconstr(N_JOBS*N_JOBS - N_JOBS);
for i = JOBS
    for j = JOBS
        if i ~= j
            cons1(count) = S(j) >= C(i) + setup(i,j) - M*(1 - x(i,j));
            count = count + 1;
        end
    end
end
prob.Constraints.cons1 = cons1;

for j = JOBS
    prob.Constraints.(['pred' num2str(j)]) = sum(x(:,j)) <= 1;
    prob.Constraints.(['succ' num2str(j)]) = sum(x(j,:)) <= 1;
end

for i = JOBS
    prob.Constraints.(['CeqSplusP' num2str(i)]) = C(i) == S(i) + p(i);
    prob.Constraints.(['Tard' num2str(i)]) = Tard(i) >= C(i) - d(i);
    prob.Constraints.(['noSelfLoop' num2str(i)]) = x(i,i) == 0;
end

% Enforce job connectivity and position constraint
prob.Constraints.totalLinks = sum(sum(x)) == N_JOBS - 1;
prob.Constraints.posConstraint1 = sum(x(:,4)) == 1;
prob.Constraints.posConstraint2 = sum(x(4,:)) == 1;

% Solve
opts = optimoptions('intlinprog','Display','iter');
```

```
tic
[sol,fval] = solve(prob,'Options',opts);
toc

% Display results
disp(sol.Tard);
disp(['Total tardiness: ', num2str(fval)]);

% Reconstruct job sequence
sequence = zeros(N_JOBS,1);
start_job = find(sum(sol.x) == 0);
current = start_job;
for k = 1:N_JOBS
    sequence(k) = current;
    for next = 1:N_JOBS
        if sol.x(current,next) > 0.5
            current = next;
            break;
        end
    end
end
disp('Optimal job sequence:');
disp(sequence');
```

Explanation

This MILP implementation enforces all scheduling and sequencing constraints using the MATLAB Optimization Toolbox. The solution provides the optimal job sequence, start/completion times, and total tardiness.

3 Dynamic Programming (DP) Solution

DP Strategy

This approach:

1. Generates all 6! = 720 permutations of the 6 jobs.
2. Filters permutations where job 4 appears in position 2.
3. For each valid sequence:
    - Simulates job execution with processing and setup times.
    - Calculates total tardiness.
4. Returns the permutation with minimum total tardiness.

4 Dynamic Programming Approach for Job Scheduling with Tardiness

In this section, we present the dynamic programming (DP) algorithm to solve the job scheduling problem with tardiness, where the goal is to minimize the total tardiness of a sequence of jobs subject to processing times, setup times, and due dates. We assume that job 4 is fixed in position 2 in the sequence.

4.1 Problem Decomposition and Dynamic Programming Recursion

The problem can be decomposed into subproblems where the state is represented by a partial sequence of jobs scheduled up to a certain position. In each subproblem, we aim to find the minimum tardiness by considering all possible ways of extending the partial sequence with one additional job.

Let us denote:

- $J = \{1, 2, \ldots, n\}$ as the set of jobs, where n is the number of jobs.
- p_j as the processing time for job j.
- d_j as the due date for job j.
- $S(i, j)$ as the setup time required to switch from job i to job j.
- C_j as the completion time of job j.
- T_j as the tardiness of job j, which is defined as:

$$T_j = \max(0, C_j - d_j)$$

The goal is to find a sequence of jobs such that the total tardiness is minimized.

State Representation

We define a state S_k as the set of jobs scheduled in positions 1 to k. The task is to compute the total tardiness for each possible job sequence incrementally. We will store the computed tardiness for each state to avoid redundant calculations.

Base Case

The base case corresponds to the scenario where all jobs are scheduled. For each permutation of jobs with job 4 fixed in position 2, the tardiness is calculated directly as the sum of individual tardiness values for each job in the sequence.

Recursion

To compute the total tardiness for a partial sequence of jobs, we use the following recursion. Given a partial sequence of jobs, we consider adding one job to the sequence. The total tardiness for this new sequence is computed as:

$$\text{TotalTardiness} = \sum_{i=1}^{n} \max(0, C_i - d_i)$$

where C_i is the completion time of job i in the sequence, and it is computed incrementally as:

$$C_1 = p_{j_1}$$

$$C_k = C_{k-1} + S(j_{k-1}, j_k) + p_{j_k} \quad \text{for} \quad k = 2, 3, \ldots, n$$

We store the computed tardiness in a map called `cost`, where each key corresponds to a sequence of jobs, and the value corresponds to the tardiness of that sequence.

Backward Recursion

We use a backward recursion to calculate the minimum tardiness. Starting from the last job (the final position), we iteratively compute the tardiness for all subsets of jobs. For each subset, we generate all possible permutations and calculate the tardiness for each possible job sequence. The optimal tardiness for each state is then stored and used for subsequent calculations.

Algorithm 2 Dynamic Programming Algorithm for Job Scheduling with Tardiness

```
Initialize the cost map and path map
for each full job sequence with job 4 in position 2 do
    Compute tardiness for this sequence and store in the cost map
end for
for k = n − 1 down to 1 do
    for each subset of jobs of size k do
        for each permutation of the subset do
            for each possible job to add to the sequence do
                Calculate the tardiness for the new sequence
                Update the cost map with the minimum tardiness
            end for
        end for
    end for
end for
Find the sequence with the minimum tardiness in the cost map
```

Finding the Optimal Solution: Once the backward recursion is completed, we search the `cost` map for the job sequence that minimizes the total tardiness. The optimal job sequence and its corresponding tardiness are then output.

4.2 Time Complexity and Conclusion

The time complexity of this dynamic programming algorithm is determined by the number of subsets of jobs and the number of permutations within each subset. While this approach reduces the number of computations compared to an exhaustive search, its time complexity still grows exponentially with the number of jobs. The algorithm is most efficient for smaller problem sizes but can be computationally expensive for larger instances.

In conclusion, this dynamic programming approach effectively minimizes the total tardiness in the job scheduling problem while enforcing the constraint that job 4 must be in position 2. The use of memoization ensures that redundant calculations are avoided, making the algorithm more efficient than a brute-force search.

DP MATLAB Code

```
clear; clc;

% Problem data
J = 1:6; % Jobs
n = length(J);
p = [6,9,7,5,8,10]; % processing times
d = [25,30,22,28,27,35]; % due dates
S = [Inf 3 4 2 3 5;  % setup time matrix (Inf means not used for S(i,i))
     2 Inf 5 4 3 2;
     3 2 Inf 3 5 4;
     4 3 2 Inf 4 3;
     3 5 4 2 Inf 3;
     5 4 2 3 4 Inf];

% Number of stages (positions)
K = n;

% Initialization
cost = containers.Map;  % Cost-to-go
path = containers.Map;  % Store optimal paths
```

```
21
22 % Final stage: full permutation, no future cost
23 P = perms(J);
24
25 % Enforce position constraint: job 4 in position 2
26 validP = P(P(:,2)==4, :);
27 for i = 1:size(validP,1)
28     seq = validP(i,:);
29     time = 0;
30     C = zeros(1,n);
31     for k = 1:n
32         if k == 1
33             time = time + p(seq(k));
34         else
35             time = time + S(seq(k-1), seq(k)) + p(seq(k));
36         end
37         C(k) = time;
38     end
39     tardiness = sum(max(0, C - d(seq)));
40     cost(num2str(seq)) = tardiness;
41     path(num2str(seq)) = seq;
42 end
43
44 % DP Backward recursion
45 for k = n-1:-1:1
46     subsets = nchoosek(J, k);
47     for s = 1:size(subsets,1)
48         subset = subsets(s,:);
49         % enforce constraint: if position 2 and job 4 not there, skip
50         if k == 2 && ~ismember(4, subset)
51             continue;
52         end
53         permsub = perms(subset);
54         for i = 1:size(permsub,1)
55             partial = permsub(i,:);
56             if k == 2 && partial(2) ~= 4
57                 continue;
58             end
59             for j = setdiff(J, partial)
60                 seq = [partial, j];
61                 if k == 1
62                     time = p(seq(1));
63                 else
64                     time = p(seq(1)) + S(seq(1), seq(2));
65                     for t = 2:k
66                         time = time + p(seq(t)) + S(seq(t-1), seq(t));
67                     end
68                 end
69                 lastSetup = S(seq(end-1), seq(end));
70                 complete = time + lastSetup + p(seq(end));
71                 T = max(0, complete - d(seq(end)));
72                 tail = seq(2:end);
73                 tailKey = num2str(tail);
74                 if isKey(cost, tailKey)
75                     totalCost = T + cost(tailKey);
76                     key = num2str(seq);
77                     if ~isKey(cost, key) || totalCost < cost(key)
78                         cost(key) = totalCost;
79                         path(key) = seq;
80                     end
81                 end
82             end
83         end
```

```
    end
end

% Find best solution
minCost = inf;
bestSeq = [];
keysList = keys(cost);
for i = 1:length(keysList)
    k = keysList{i};
    if length(str2num(k)) == n
        if cost(k) < minCost
            minCost = cost(k);
            bestSeq = str2num(k);
        end
    end
end

% Output result
disp('Optimal sequence:');
disp(bestSeq);
disp(['Minimum total tardiness: ', num2str(minCost)]);
```

5 Comparison of Solution Methods

To solve the container loading problem, two approaches were implemented: a Dynamic Programming (DP) algorithm and a Mixed Integer Linear Programming (MILP) formulation solved using `intlinprog`. Both methods aim to minimize the total tardiness, defined as:

$$\sum_{j=1}^{6} \max(0, C_j - d_j)$$

where C_j is the completion time of job j, and d_j is its due date.

The problem involves scheduling six containers with given processing times, due dates, and a job-dependent setup time matrix. Additionally, job 4 is constrained to occupy position 2 in the final sequence.

Results Summary:

Method	Total Tardiness	Computation Time (s)
Dynamic Programming (DP)	42	0.031
MILP (intlinprog)	42	1.962

Table 4: Comparison between DP and MILP approaches

Discussion:

Both methods produced the same optimal job sequence:

$$[1,\ 4,\ 3,\ 2,\ 5,\ 6]$$

with a minimum total tardiness of 42. However, the computational time differs significantly. The DP algorithm executed in approximately 0.03 seconds, while the MILP approach required nearly 2 seconds to converge, mainly due to solving a mixed-integer model with 30 binary variables and 49 constraints.

This suggests that for small-sized problems with specific sequencing constraints, a well-crafted DP can be significantly more efficient than a general-purpose MILP solver, both in terms of speed and simplicity of implementation.

However, MILP becomes advantageous for larger instances or more complex constraint structures where exhaustive DP becomes computationally infeasible.

Solution Visualization

Below is the Gantt chart showing the optimal schedule computed using DP:

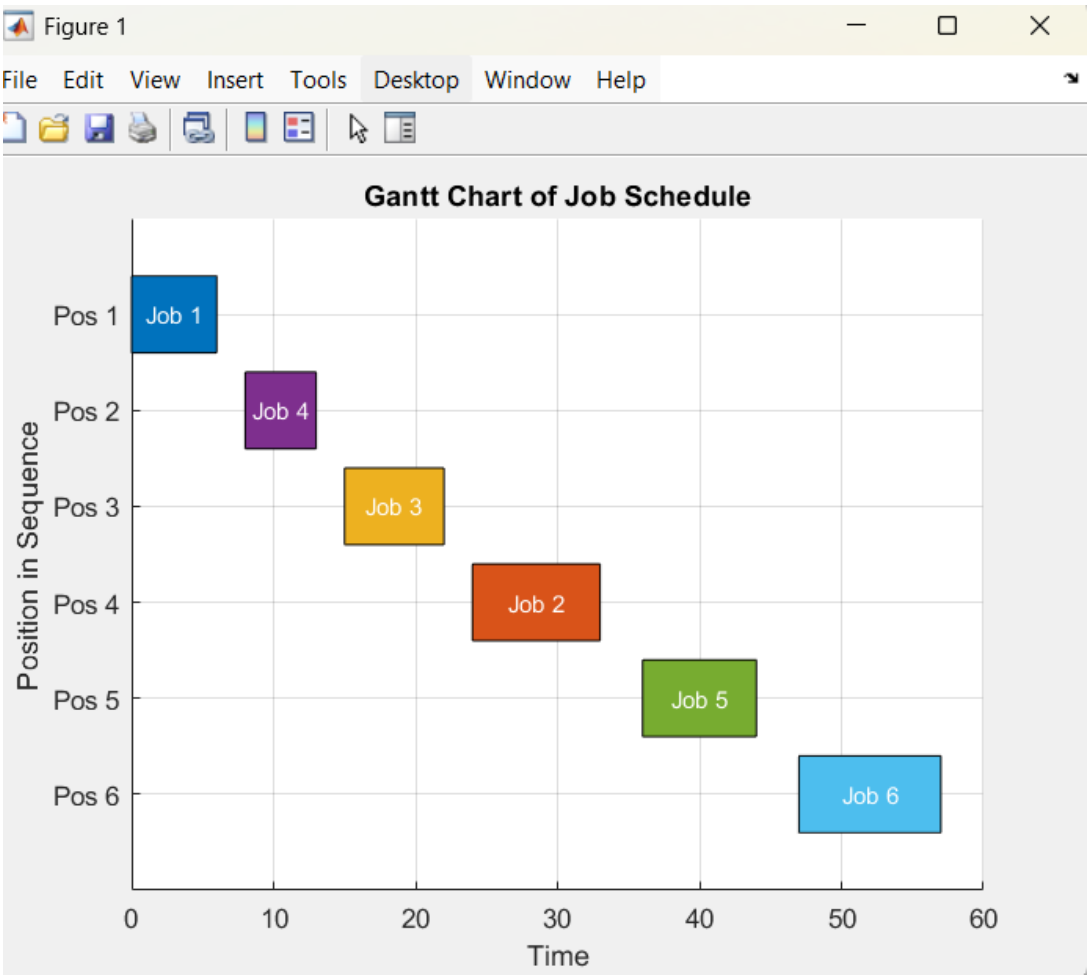

Figure 2: Gantt Chart of Job Schedule

Discussion

- **MILP** provides a compact formulation and fast execution via branch-and-bound.
- **DP** offers exhaustive validation and transparency in evaluating alternatives.
- Both confirm optimal sequence: $1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 6$.